

Expressions and Operators

1

Expressions

An *expression* is any valid unit of code that resolves to a value. Every syntactically valid expression resolves to some value.

The expression `x = 7` is an example of the first type. This expression uses the `=` operator to assign the value seven to the variable `x`. The expression itself evaluates to seven.

The code `3 + 4` is an example of the second expression type. This expression uses the `+` operator to add three and four together without assigning the result, seven, to a variable.

Operators

JavaScript operators are special symbols that perform operations on one or more operands (values).

For example:

`2 + 3;`

Here, we used the `+` operator to add the operands **2** and **3**.

JavaScript Operator Types

Here is a list of different JavaScript operators you will learn in this tutorial:

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators

4. Logical Operators
5. Bitwise Operators
6. String Operators

1. JavaScript Arithmetic Operators

We use arithmetic operators to perform **arithmetic calculations** like addition, subtraction, etc. For example,

5 - 3;

Here, we used the - operator to subtract **3** from **5**.

Commonly Used Arithmetic Operators

Operator	Name	Example
+	Addition	3 + 4 // 7
-	Subtraction	5 - 3 // 2
*	Multiplication	2 * 3 // 6
/	Division	4 / 2 // 2
%	Remainder	5 % 2 // 1
++	Increment (increments by 1)	++5 or 5++ // 6
--	Decrement (decrements by 1)	--4 or 4-- // 3
**	Exponentiation (Power)	4 ** 2 // 16

2. JavaScript Assignment Operators

We use assignment operators to **assign** values to variables. For example,

```
let x = 5;
```

Here, we used the = operator to assign the value **5** to the variable x.

Commonly Used Assignment Operators

Operator	Name	Example
=	Assignment Operator	a = 7;
+=	Addition Assignment	a += 5; // a = a + 5
-=	Subtraction Assignment	a -= 2; // a = a - 2
*=	Multiplication Assignment	a *= 3; // a = a * 3
/=	Division Assignment	a /= 2; // a = a / 2
%=	Remainder Assignment	a %= 2; // a = a % 2
=	Exponentiation Assignment	a **= 2; // a = a2

3. JavaScript Comparison Operators

We use comparison operators to **compare** two values and return a boolean value (true or false). For example,

```
const a = 3, b = 2;
```

```
console.log(a > b);
```

```
// Output: true
```

Here, we have used the > comparison operator to check whether a (whose value is **3**) is greater than b (whose value is **2**).

Since **3** is greater than **2**, we get true as output.

Note: In the above example, $a > b$ is called a **boolean expression** since evaluating it results in a boolean value.

Commonly Used Comparison Operators

4

Operator	Meaning	Example
<code>==</code>	Equal to	<code>3 == 5</code> gives us false
<code>!=</code>	Not equal to	<code>3 != 4</code> gives us true
<code>></code>	Greater than	<code>4 > 4</code> gives us false
<code><</code>	Less than	<code>3 < 3</code> gives us false
<code>>=</code>	Greater than or equal to	<code>4 >= 4</code> gives us true
<code><=</code>	Less than or equal to	<code>3 <= 3</code> gives us true
<code>===</code>	Strictly equal to	<code>3 === "3"</code> gives us false
<code>!==</code>	Strictly not equal to	<code>3 !== "3"</code> gives us true

Difference between equality (`==` and `!=`) and strict equality (`===` and `!==`) operators.

The equality operators (`==` and `!=`) convert both operands to the same type before comparing their values. For example,

```
console.log(3 == "3"); // true
```

Here, we used the `==` operator to compare the number **3** and the string 3.

By default, JavaScript converts string 3 to number **3** and compares the values.

However, the strict equality operators (`===` and `!==`) do not convert operand types before comparing their values. For example,

```
console.log(4 === "4"); // false
```

Here, JavaScript didn't convert string 4 to number **4** before comparing their values.

Thus, the result is false, as number **4** isn't equal to string 4.

4. JavaScript Logical Operators

We use logical operators to perform logical operations on boolean expressions. For example,

```
const x = 5, y = 3;
```

```
console.log((x < 6) && (y < 5));
```

```
// Output: true
```

Here, `&&` is the logical operator **AND**. Since both `x < 6` and `y < 5` are true, the combined result is true.

Commonly Used Logical Operators

Operator	Syntax	Description
&& (Logical AND)	expression1 && expression2	true only if both expression1 and expression2 are true
 (Logical OR)	expression1 expression2	true if either expression1 or expression2 is true
! (Logical NOT)	!expression	false if expression is true and vice versa

Example

```
let x = 3;  
  
// logical AND  
console.log((x < 5) && (x > 0)); // true  
console.log((x < 5) && (x > 6)); // false
```

```
// logical OR  
console.log((x > 2) || (x > 5)); // true  
console.log((x > 3) || (x < 0)); // false
```

```
// logical NOT  
console.log(!(x == 3)); // false  
console.log(!(x < 2)); // true
```

Note: We use comparison and logical operators in decision-making and loops. You will learn about them in detail in later tutorials.

5. JavaScript Bitwise Operators

We use bitwise operators to perform **binary operations** on integers.

Operator	Description	Example
&	Bitwise AND	5 & 3 // 1
	Bitwise OR	5 3 // 7

<code>^</code>	Bitwise XOR	<code>5 ^ 3 // 6</code>
<code>~</code>	Bitwise NOT	<code>~5 // -6</code>
<code><<</code>	Left shift	<code>5 << 1 // 10</code>
<code>>></code>	Sign-propagating right shift	<code>-10 >> 1 // -5</code>
<code>>>></code>	Zero-fill right shift	<code>-10 >>> 1 // 2147483643</code>

Note: We rarely use bitwise operators in everyday programming.

6. JavaScript String Concatenation Operator

In JavaScript, you can also use the `+` operator to concatenate (join) two strings. For example,

```
let str1 = "Hel", str2 = "lo";
```

```
console.log(str1 + str2);
```

```
// Output: Hello
```

Here, we used the `+` operator to concatenate `str1` and `str2`.